Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №5

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 5
Листинг: 6
Main.c 6
Student.h 6
Student.c 6
List.h 7
List.c 7
Makefile 8

Задание: разделите программу лабораторной 1 на модули. Реализуйте функции работы со списками в библиотеке совместного доступа .so.

Цель: получение навыков разработки библиотек динамической компоновки на платформе Linux.

Ход работы:

Разделим программу первой Лабораторной работы на модули. После разделения программа приняла следующий вид (содержимое файлов см. Листинг):

- main.c
- list.h (заголовки функций insertStudent, deleteStudent, displayStudents, serializeStudents);
- list.c (реализация функций insertStudent, deleteStudent, displayStudents, serializeStudents);
- student.h (структура Student и заголовок функции createStudent);
- student.c (реализация функции createStudent).

Для компиляции программы используются следующие команды:

1. gcc -c -fPIC student.c -o student.o
2. gcc -c -fPIC list.c -o list.o
3. gcc -shared -o libstudents.so student.o list.o
4. gcc -o main main.c -L. -lstudents

Флаг -fPIC (Position Independent Code) используется при компиляции программного кода для создания исполняемых файлов и библиотек с позиционно-независимым кодом. Это важное требование при создании shared-библиотек (динамических).

Позиционно-независимый код (PIC) позволяет коду загружаться и выполняться в памяти независимо от конкретного адреса загрузки. Это важно, потому что разделяемые библиотеки загружаются в адресное пространство процесса во время выполнения, и конкретное местоположение библиотеки может меняться при каждой загрузке.

Для удобства компиляции нами был написан makefile (код приложен в Листинг).

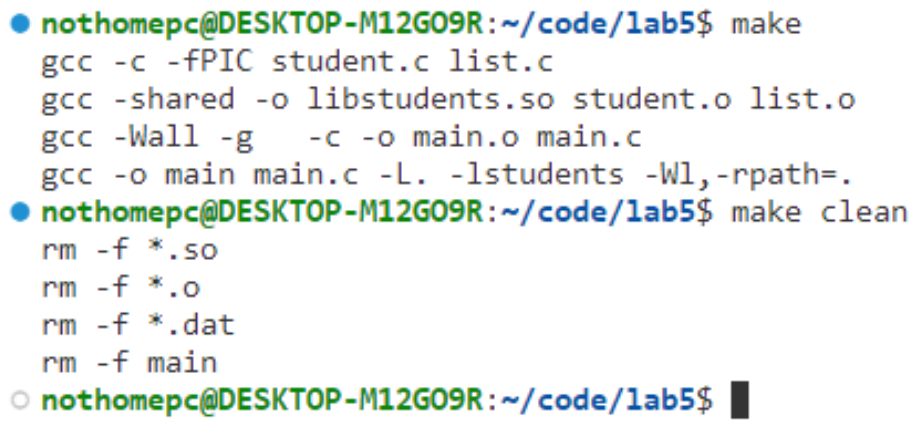


Рис. 1 – Пример работы makefile.

Вывод:

Нами была разделена программа лабораторной работы №1 на модули student, list и main; написан makefile для удобства компиляции программы. Реализованы функции работы со списками в библиотеке совместного доступа .so.

Листинг:

Main.c

```c
#include <stdlib.h>
#include "student.h"
#include "list.h"

int main() {
    struct Student* student1 = createStudent("Иван", 101, 85.5);
    struct Student* student2 = createStudent("Мария", 102, 92.0);
    struct Student* student3 = createStudent("Пётр", 103, 78.2);

    insertStudent(student1);
    insertStudent(student2);
    insertStudent(student3);

    displayStudents();
    deleteStudent(102);
    displayStudents();

    serializeStudents("journal.dat");
    return 0;
}
```

Student.h

```c
// student.h
#ifndef STUDENT_H
#define STUDENT_H

struct Student {
    char name[50];
    int roll_number;
    float grade;
    struct Student* next;
};

struct Student* createStudent(char name[], int roll_number, float grade);

#endif
```

Student.c

```c
#include "student.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Student* createStudent(char name[], int roll_number, float grade) {
    struct Student* newStudent = (struct Student*)malloc(sizeof(struct Student));
    if (newStudent != NULL) {
        strncpy(newStudent->name, name, sizeof(newStudent->name));
        newStudent->roll_number = roll_number;
```

```
        newStudent->grade = grade;
        newStudent->next = NULL;
    }
    return newStudent;
}
```

List.h

```
// list.h
#ifndef LIST_H
#define LIST_H

#include "student.h"

void insertStudent(struct Student* newStudent);
void deleteStudent(int roll_number);
void displayStudents();
void serializeStudents(const char *filename);

#endif
```

List.c

```
#include "student.h"
#include "list.h"
#include <stdio.h>
#include <stdlib.h>

struct Student* head = NULL;

void insertStudent(struct Student* newStudent) {
    newStudent->next = head;
    head = newStudent;
    printf("Студент добавлен в журнал.\n");
}

void deleteStudent(int roll_number) {
    struct Student* current = head;
    struct Student* prev = NULL;
    while (current != NULL) {
        if (current->roll_number == roll_number) {
            if (prev == NULL) {
                head = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            printf("Студент удален из журнала.\n");
            return;
        }
        prev = current;
        current = current->next;
```

```
    }
    printf("Студент с номером %d не найден.\n", roll_number);
}

void displayStudents() {
    struct Student* current = head;
    if (current == NULL) {
        printf("\nЖурнал успеваемости пуст.\n");
    } else {
        printf("\nЖурнал успеваемости студентов:\n");
        while (current != NULL) {
            printf("Имя: %s, Номер: %d, Оценка: %.2f\n", current->name, current-
>roll_number, current->grade);
            current = current->next;
        }
    }
}

void serializeStudents(const char *filename)
{
    FILE *file = fopen(filename, "wb");
    if (file == NULL)
    {
        perror("Ошибка при открытии файла для записи");
        return;
    }
    struct Student *current = head;
    while (current != NULL)
    {
        fwrite(current, sizeof(struct Student), 1, file);
        current = current->next;
    }
    fclose(file);
    printf("Сериализация завершена.\n");
}
```

Makefile

```
CC = gcc

CFLAGS = -Wall -g

SRC = student.c list.c
OBJ = $(SRC:.c=.o)

TARGET = main

LIBNAME = libstudents.so

$(TARGET): $(LIBNAME) main.o
    gcc -o main main.c -L. -lstudents -Wl,-rpath=.
```

```
$(LIBNAME): $(OBJ)
    gcc -shared -o libstudents.so $(OBJ)

$(OBJ):
    $(CC) -c -fPIC $(SRC)

clean:
    rm -f *.so
    rm -f *.o
    rm -f *.dat
    rm -f main
.PHONY: clean

all: $(TARGET)
```